

Day 15: Decision Trees

Tennis Play Prediction with Decision Trees

Sandesh Dhital

Nesfield International College

August 6, 2026

What is a Decision Tree?

A Decision Tree is a **flowchart-like** structure used for classification and regression.

- Each **internal node** = a question about a feature
- Each **branch** = an answer to that question
- Each **leaf node** = a final prediction (class label)

Example

Is Outlook Sunny? → Yes → Is Humidity High? → Yes → **Don't Play**

Key advantage: Easy to understand and interpret — even non-technical people can follow the logic!

The Tennis Dataset

A classic dataset with 14 examples of weather conditions and whether tennis was played.

Features	Target
Outlook (Sunny, Overcast, Rain)	PlayTennis
Temperature (Hot, Mild, Cool)	Yes (9 days)
Humidity (High, Normal)	No (5 days)
Wind (Weak, Strong)	

Goal: Build a tree that predicts whether to play tennis based on weather conditions.

Entropy: Measuring Disorder

Entropy tells us how “mixed up” or impure a set of data is.

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- $H = 0$: All samples belong to **one class** (perfectly pure)
- $H = 1$: Samples are **equally split** (maximum impurity for 2 classes)

For our dataset:

- PlayTennis: 9 Yes, 5 No out of 14 total
- $H = -(9/14) \log_2(9/14) - (5/14) \log_2(5/14) = 0.940$

Information Gain: Picking the Best Feature

Information Gain = how much entropy **decreases** after splitting on a feature.

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

For the Tennis dataset:

Feature	Information Gain
Outlook	0.2467 (highest!)
Humidity	0.1518
Wind	0.0481
Temperature	0.0292

⇒ **Outlook** becomes the root node because it gives the most information!

Building the Tree: ID3 Algorithm

- 1 Calculate entropy of the whole dataset
- 2 For each feature, calculate Information Gain
- 3 Split on the feature with the **highest IG**
- 4 Repeat for each branch until:
 - All samples in a branch are the same class (pure leaf), OR
 - No more features to split on

For our tree:

- Root: **Outlook**
- Outlook = Overcast → Always Yes (pure leaf!)
- Outlook = Sunny → Split further on Humidity
- Outlook = Rain → Split further on Wind

Overfitting in Decision Trees

Decision Trees can easily **overfit** by memorizing every detail in the training data.

Problem	Solution
Tree grows too deep	Set <code>max_depth</code> to limit levels
Too many leaves	Set <code>min_samples_leaf</code> to require minimum samples per leaf
Splits on noise	Set <code>min_samples_split</code> to require minimum samples before splitting
Post-hoc fix	Pruning : grow full tree, then remove branches that don't improve validation accuracy

Splitting Criteria: Entropy vs Gini

Scikit-learn offers two ways to measure impurity:

Entropy (ID3/C4.5)

$$H = - \sum p_i \log_2(p_i)$$

- Range: $[0, 1]$ for binary
- Slightly slower (log calculation)
- Tends to produce more balanced trees

Gini Impurity (CART)

$$Gini = 1 - \sum p_i^2$$

- Range: $[0, 0.5]$ for binary
- Faster to compute
- Default in scikit-learn

In practice, both give very similar results. We use **entropy** here to match the ID3 theory.

Decision Trees: Pros and Cons

Advantages

- Easy to understand and visualize
- No feature scaling needed
- Handles both numerical and categorical data
- Fast prediction
- Shows feature importance

Disadvantages

- Prone to overfitting
- Unstable (small data change → different tree)
- Biased toward features with many values
- Cannot capture diagonal boundaries
- Greedy (locally optimal, not globally)

Decision Tree vs KNN vs Logistic Regression

	Decision Tree	KNN	Logistic Reg.
Learning	Builds rules	Memorizes data	Learns weights
Scaling needed	No	Yes	Recommended
Interpretability	Very high	Low	Medium
Overfitting risk	High	Medium	Low
Decision boundary	Rectangular	Flexible	Linear
Training speed	Fast	None	Medium
Prediction speed	Fast	Slow	Fast

Summary

- Decision Trees split data by asking **yes/no questions** about features
- **Entropy** measures disorder; **Information Gain** picks the best split
- The **ID3 algorithm** greedily picks the highest IG feature at each node
- Trees are highly **interpretable** — you can read the decision rules directly
- Control overfitting with `max_depth`, `min_samples_leaf`, or pruning
- No feature scaling required (unlike KNN)

Next Steps

Random Forests — combine many decision trees to reduce overfitting and improve accuracy!